Градиентный бустинг

Паточенко Евгений

НИУ ВШЭ

План занятия

- Напоминание
- Бустинг
- XGBoost
- CatBoost
- LightGBM

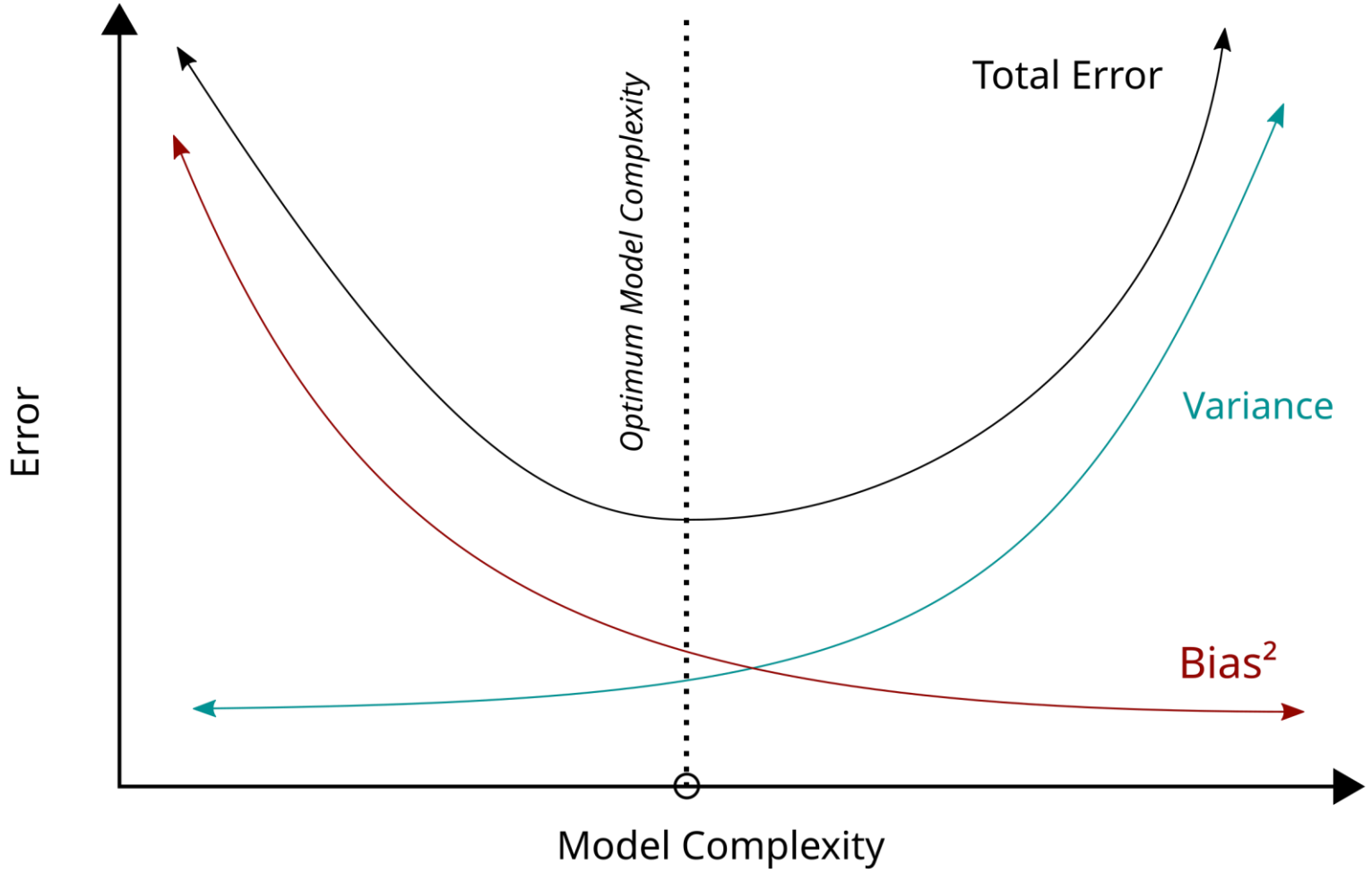
Напоминание: декомпозиция ошибки

Компоненты ошибки:

- Смещение (bias) — способность модели приблизить лучшую среди всех возможных моделей
- Разброс (variance) — устойчивость модели к изменениям в обучающей выборке
- Шум (noise) — характеристика сложности и противоречивости данных

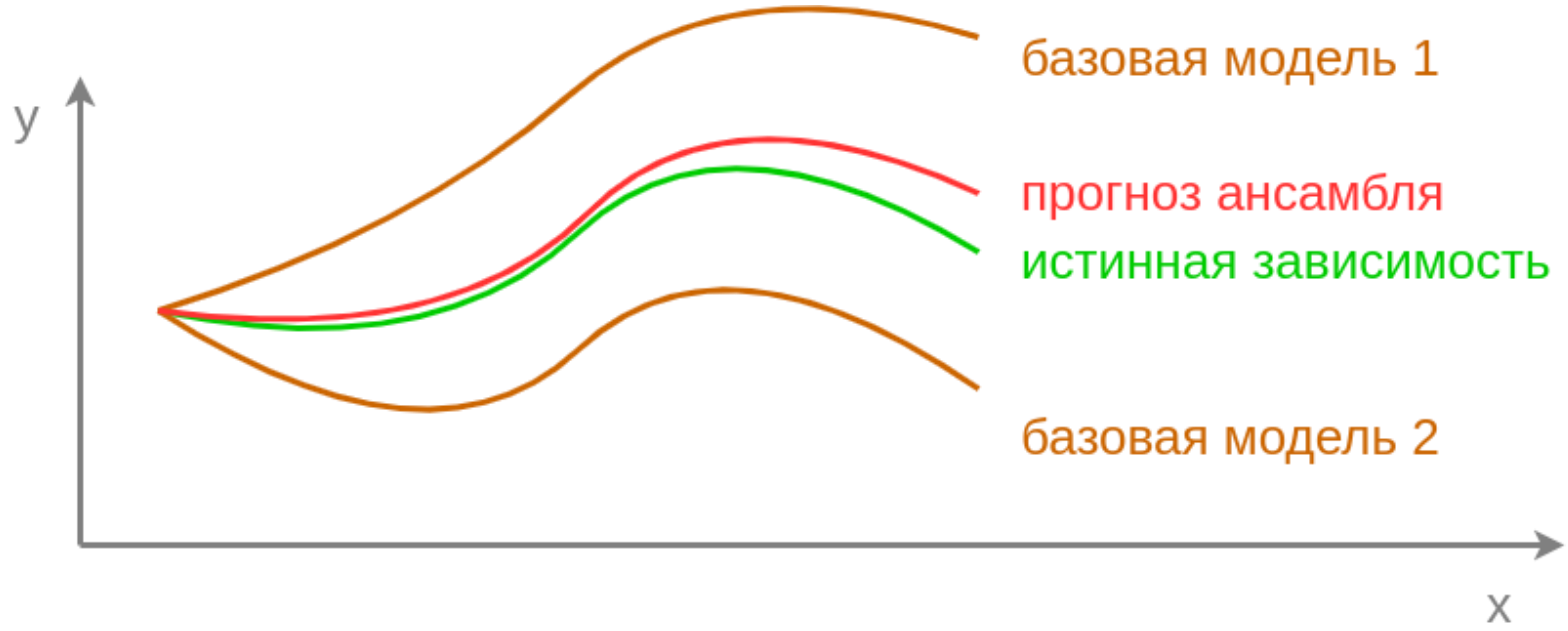
$$Q(a) = \mathbb{E}_x \text{bias}_X^2 a(x, X) + \mathbb{E}_x \mathbb{V}_X[a(x, X)] + \sigma^2$$

Напоминание: декомпозиция ошибки



Источник: https://en.wikipedia.org/wiki/Bias%E2%80%93variance_tradeoff

Напоминание: ансамбли и бэггинг



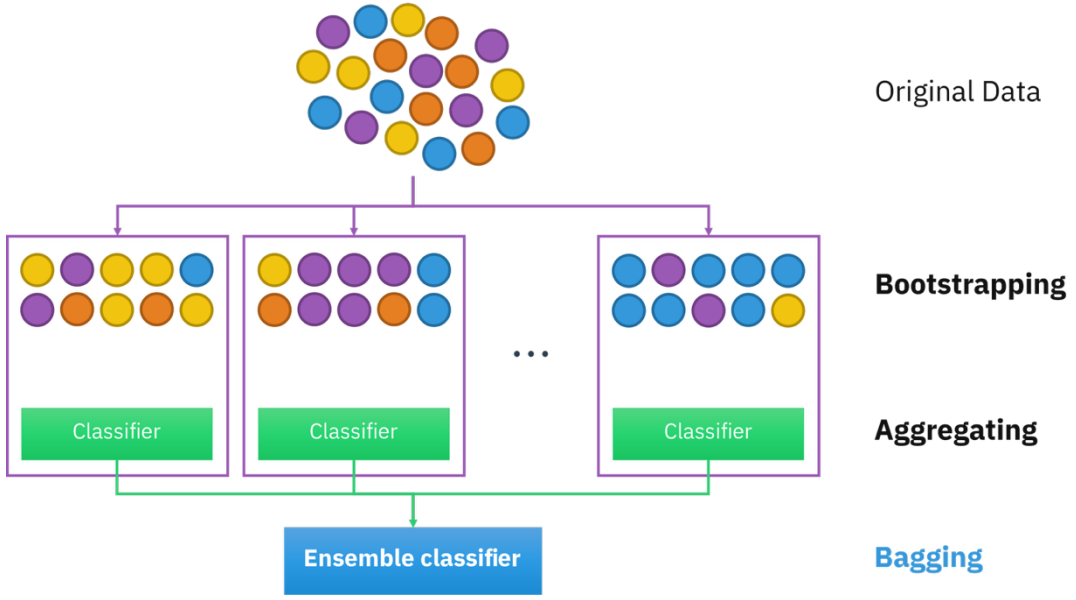
Источник: https://deepmachinelearning.ru/docs/Machine-learning/Model-ensembles/Model-ensembles

Бэггинг снижает разброс, смещение снижает бустинг

Напоминание: ансамбли и бэггинг

Бэггинг (bagging, bootstrap aggregating) — метод, при котором несколько моделей обучаются независимо на случайных подвыборках данных:

1. Создаются бутстреп-выборки
2. Обучаются одинаковые модели
3. Предсказания усредняются или голосуются

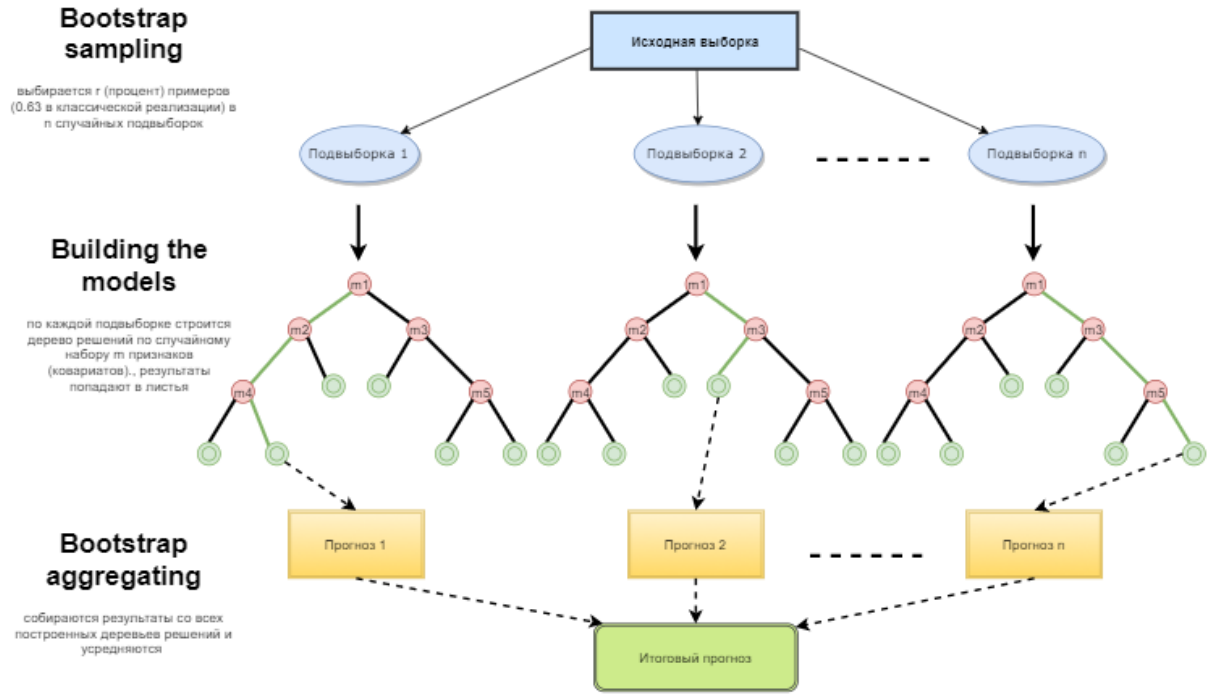


Источник: https://en.wikipedia.org/wiki/Bootstrap_aggregating

Напоминание: случайный лес

Базовые модели — деревья решений. На каждом шаге обучения используется дополнительное случайное подмножество признаков для разделения узлов:

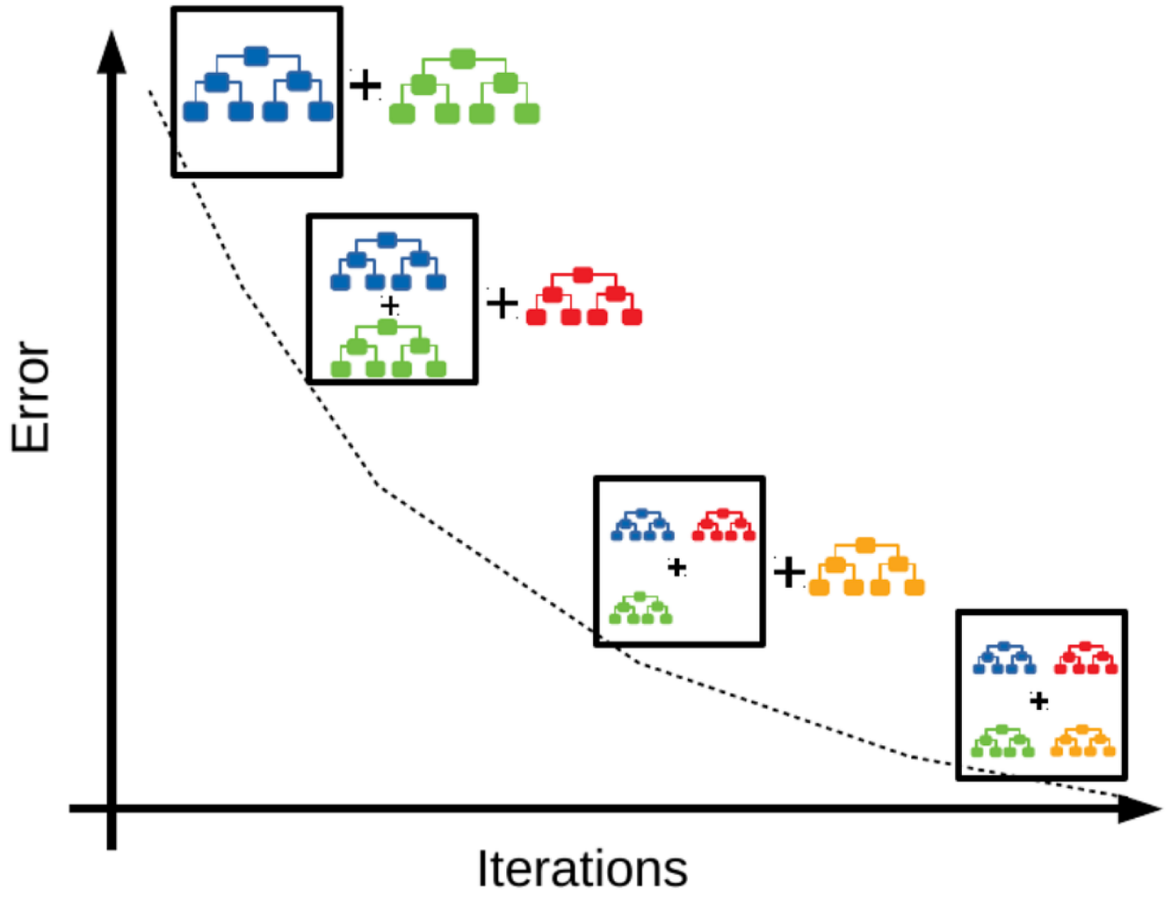
1. Создаются бутстреп-выборки
2. Обучаются несколько деревьев
3. При построении каждого узла дерева выбирается случайное подмножество признаков, из которых выбирается лучший сплит
4. Предсказания усредняются или голосуются



Источник: https://www.mql5.com/ru/articles/3856

Бустинг

Набор алгоритмов, каждый из которых исправляет ошибки предыдущего



Источник: https://medium.com/@hemashreekilari9/understanding-gradient-boosting-632939b98764

Бустинг в задаче регрессии

Решаем задачу регрессии с минимизацией квадратичной ошибки:

$$\frac{1}{2}\sum_{i=1}^{l}(a(x_i)-y_i)^2 \to \min_a$$

Ищем алгоритм $a(x)$ в виде суммы N базовых алгоритмов

$$a(x)=\sum_{n=1}^{N} b_n(x),$$

где базовые алгоритмы $b_n(x)$ принадлежат некоторому семейству A

Бустинг в задаче регрессии

1. Ищем алгоритм $b_1(x)$, минимизирующий ошибку:

$$b_1(x) = \underset{b \in A}{\operatorname{argmin}} \frac{1}{2} \sum_{i=1}^{l} (b(x_i) - y_i)^2$$

Ошибка на объекте x:

$$s = y - b_1(x),$$

Бустинг в задаче регрессии

1. Ищем алгоритм $b_1(x)$, минимизирующий ошибку:

$$b_1(x) = \underset{b \in A}{\operatorname{argmin}} \frac{1}{2} \sum_{i=1}^{l} (b(x_i) - y_i)^2$$

Ошибка на объекте x:

$$s = y - b_1(x),$$

Целевая переменная для следующего алгоритма — вектор ошибок s (а не исходный вектор y)

Бустинг в задаче регрессии

1. Ищем алгоритм $b_1(x)$, минимизирующий ошибку:

$$b_1(x) = \underset{b \in A}{\operatorname{argmin}} \frac{1}{2} \sum_{i=1}^{l} (b(x_i) - y_i)^2$$

2. Ищем алгоритм $b_2(x)$, настраивающийся на ошибки s первого алгоритма:

$$b_2(x) = \underset{b \in A}{\operatorname{argmin}} \frac{1}{2} \sum_{i=1}^{l} (b(x_i) - \boldsymbol{s}_i)^2$$

Бустинг в задаче регрессии

Каждый следующий алгоритм настраиваем на ошибку предыдущих

Ошибка:

$$s_i^{(N)} = y_i - \sum_{n=1}^{N-1} b_n(x_i) = y_i - a_{N-1}(x_i)$$

Алгоритм $b_N(x)$:

$$b_N(x) = \underset{b \in A}{\operatorname{argmin}} \frac{1}{2} \sum_{i=1}^{l} (b(x_i) - \boldsymbol{s_i^{(N)}})^2$$

Градиентный бустинг

Ошибка на N-м шаге — это **антиградиент** функции потерь по ответу модели, вычисленный в точке ответа уже построенной композиции:

$$s_i^{(N)} = y_i - a_{N-1}(x_i) = -\frac{\partial}{\partial z}\frac{1}{2}(z - y_i)^2, \quad z = a_{N-1}(x_i)$$

Градиентный бустинг: алгоритм

Берем дифференцируемую функцию потерь $L(y, z)$ и начальное приближение a_0 (константа; для MSE это среднее ответов).

Для $N = 1, 2, \ldots$:

1. Считаем антиградиент в точке ответов композиции:

$s_i^{(N)} = -\partial L(y_i, z)/\partial z$ при $z = a_{N-1}(x_i)$

2. Обучаем b_N на «ответах» $s_i^{(N)}$ обычной MSE:

$b_N = \operatorname{argmin}_b \frac{1}{2} \Sigma_i (b(x_i) - s_i^{(N)})^2$

3. Обновляем композицию:

$a_N(x) = a_{N-1}(x) + \eta \cdot b_N(x)$, где $\eta \in (0, 1]$ — темп обучения

Бустинг в классификации

Метки $y \in \{-1, +1\}$, потеря логистическая: $L(y, z) = \ln(1 + \exp(-yz))$

Антиградиент: $s_i^{(N)} = y_i / (1 + \exp(y_i \cdot a_{N-1}(x_i)))$

Деревья b_N остаются регрессионными: они предсказывают числа s_i, а не классы

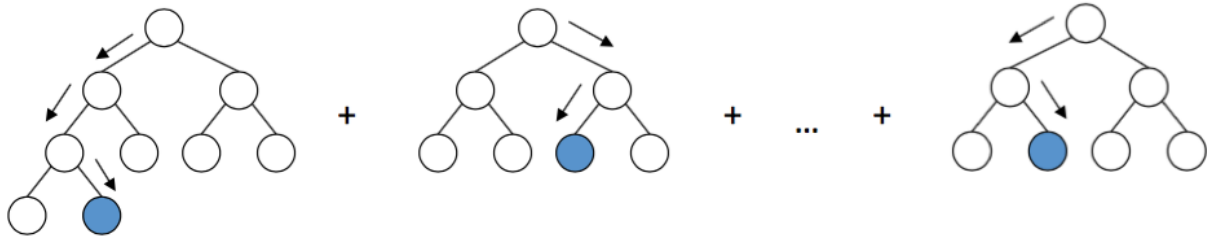
Ответ композиции $a(x)$ — это не вероятность, а логит. Вероятность класса +1:

$P(y = +1 \mid x) = 1 / (1 + \exp(-a(x)))$

Чем сильнее композиция ошибается на объекте, тем больше $|s_i|$ и тем больше внимания он получит в следующем дереве

Градиентный бустинг

Как и в других ансамблях, в бустинге в качестве базовой модели чаще всего используются **решающие деревья (градиентный бустинг над решающими деревьями, Gradient Boosting on Decision Trees, GBDT)**



Можно использовать и линейные модели, но тогда алгоритм будет представлять линейную комбинацию линейных моделей (то есть будет оставаться линейной моделью) и не сможет эффективно работать с нелинейными зависимостями

Градиентный бустинг

Оценка градиента деревьями решений

При построении следующего решающего дерева для оценки качества приближения вектора антиградиента g на i-ом объекте используют следующие функции:

$$L_2(g,p) = \sum_{i=1}^{l} (p_i - g_i)^2 ,$$

$$Cosine(g,p) = -\frac{\sum_{i=1}^{l}(p_i \cdot g_i)}{\sqrt{\sum_{i=1}^{l} {p_i}^2} \cdot \sqrt{\sum_{i=1}^{l} {g_i}^2}}$$

где p_i — предсказание антиградиента на i-ом объекте, а g_i — его истинное значение. Cosine сравнивает только направления векторов и не зависит от их длины

Градиентный бустинг

Слишком простые базовые алгоритмы плохо приближают антиградиент функции потерь, то есть градиентный бустинг может свестись к случайному блужданию

Слишком сложные базовые алгоритмы приводят к переобучению, то есть бустинг подгонится под обучающую выборку за несколько шагов.

Градиентный бустинг

Слишком простые базовые алгоритмы плохо приближают антиградиент функции потерь, то есть градиентный бустинг может свестись к случайному блужданию

Слишком сложные базовые алгоритмы приводят к переобучению, то есть бустинг подгонится под обучающую выборку за несколько шагов.

Для решающих деревьев чаще всего берут глубину от 3 до 8, точное значение зависит от задачи.

Градиентный бустинг

Слишком простые базовые алгоритмы плохо приближают антиградиент функции потерь, то есть градиентный бустинг может свестись к случайному блужданию

Слишком сложные базовые алгоритмы приводят к переобучению, то есть бустинг подгонится под обучающую выборку за несколько шагов.

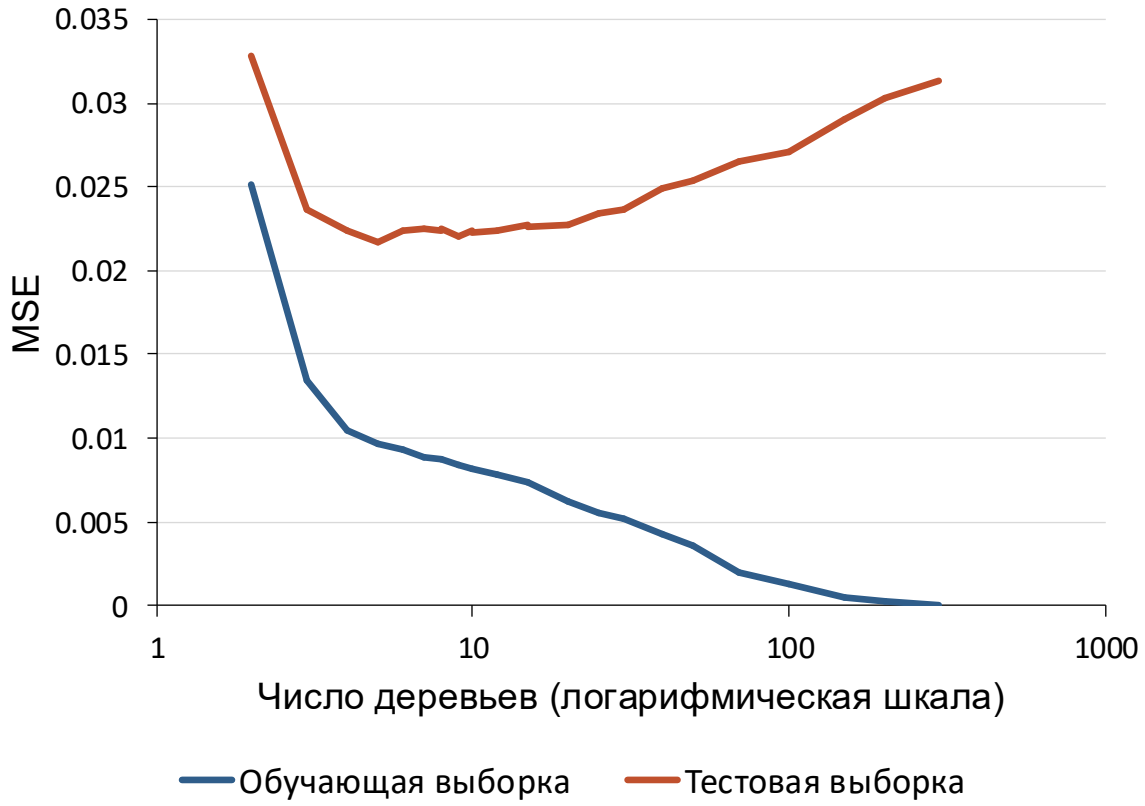
Можно также сократить шаг обучения. Чем меньше темп η, тем меньше степень доверия к каждому базовому алгоритму и тем лучше качество итоговой композиции, но тем больше деревьев нужно.

Сколько деревьев брать

Ошибка на обучении падает с каждым деревом. Ошибка на валидации сначала падает, потом растет.

Ранняя остановка (early stopping): следим за валидационной ошибкой и останавливаемся, если она не улучшалась k итераций подряд.

Параметр n_estimators задает только верхнюю границу. Чем меньше η, тем больше деревьев нужно.



Игрушечный пример с семинара: глубина деревьев 2, $\eta = 0{,}5$

Стохастический градиентный бустинг

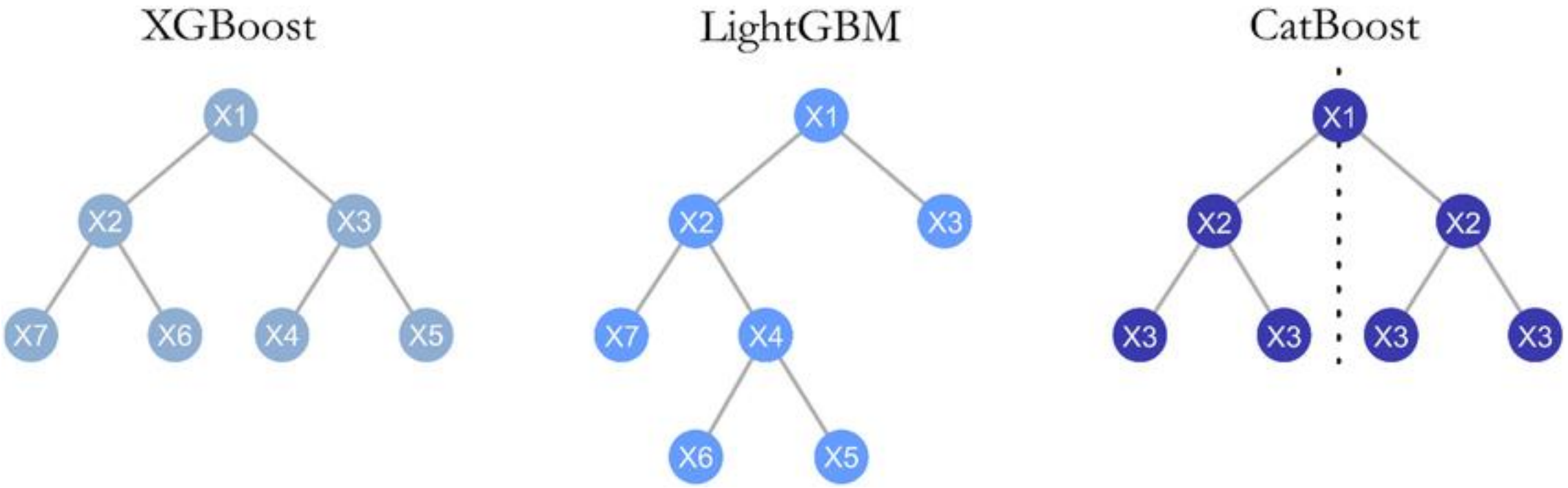
Аналогично логике градиентного спуска: обучаем базовый алгоритм b_N не на всей выборке X, а на случайной подвыборке $X^k \subset X$

✓Деревья меньше похожи друг на друга, переобучение снижается

✓ Вычисления становятся быстрее

Обычно берут от 50% до 80% выборки, например $\left|X^k\right| = \frac{1}{2}|X|$. В sklearn и XGBoost по умолчанию подвыборка не используется

Реализации градиентного бустинга



Источник: https://www.researchgate.net/figure/Different-tree-structures-and-split-indexes-shown-inside-each-node-generated-by_fig1_373472181

XGBoost (eXtreme Gradient Boosting)

- Базовый алгоритм приближает направление, посчитанное с учетом второй производной функции потерь
- Функционал регуляризуется — добавляются штрафы за количество листьев и за норму коэффициентов
- При построении дерева используется критерий информативности, зависящий от оптимального вектора сдвига
- Критерий останова также зависит от оптимального сдвига

$$\sum_{i=1}^{l}(-s_i b(x_i) + \frac{1}{2} h_i b^2(x_i)) + \gamma J + \frac{\lambda}{2}\sum_{j=1}^{J} b_j^2 \to \min_b$$

XGBoost (eXtreme Gradient Boosting)

$g_i = -s_i$ — градиент потерь на объекте i, $h_i = \partial^2 L / \partial z^2$ — вторая производная

J — число листьев, b_j — значение в листе j

γ — штраф за каждый лист, λ — штраф за квадрат значений в листьях

Если структура дерева задана, оптимальное значение в листе j:

$b_j^* = -G_j / (H_j + \lambda)$, $G_j = \Sigma\, g_i$, $H_j = \Sigma\, h_i$ по объектам листа

Это шаг Ньютона: числитель задает направление, знаменатель — длину шага. Чем больше λ, тем осторожнее шаг.

XGBoost (eXtreme Gradient Boosting)

- **Базовый алгоритм приближает направление, посчитанное с учетом второй производной функции потерь**

Первая производная говорит в каком направлении уменьшать ошибку

Вторая производная (кривизна) говорит, как быстро меняется сам градиент

То есть благодаря второй производной алгоритм знает не только в каком направлении делать шаг, но и оптимальный размер шага

XGBoost (eXtreme Gradient Boosting)

- **Базовый алгоритм приближает направление, посчитанное с учетом второй производной функции потерь**

Это позволяет:

— улучшить точность выбора разбиений дерева,

— считать оптимальное значение в листе (шаг Ньютона),

— быстрее сходиться: нужно меньше деревьев,

— уменьшать переобучение за счет штрафов γ и λ,

— стабилизировать обучение на сложных функциях потерь.

CatBoost (Categorical Boosting)

Алгоритм, разработанный в Яндексе. Независимая реализация градиентного бустинга над деревьями со своими приемами:

— умеет обрабатывать категориальные признаки

— использует симметричные деревья решений с одним и тем же решающим правилом на одном и том же уровне

— при кодировании категориальных признаков используется набор методов: OHE, счетчики, комбинации признаков и т.д.

— работает с пропусками в данных

CatBoost (Categorical Boosting)

Алгоритм, разработанный в Яндексе. Независимая реализация градиентного бустинга над деревьями со своими приемами:

— быстро применяется благодаря симметричным деревьям, есть обучение на GPU

— показывает хороший результат даже без тонкой настройки гиперпараметров

— имеет встроенную кросс-валидацию

— имеет детекцию переобучения

— имеет встроенное вычисление значений метрик

CatBoost: категории и ordered boosting

Если заменить категорию средним ответом по ней, учитывая и сам объект, возникает утечка: признак «знает» ответ объекта.

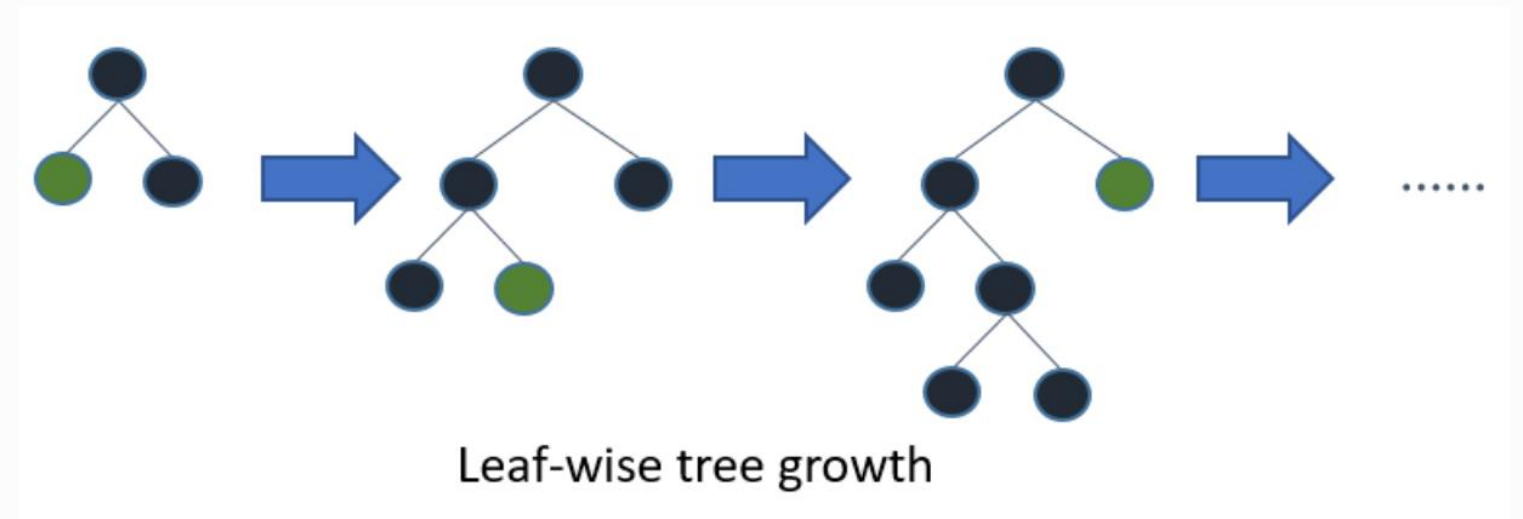
CatBoost считает такую статистику только по объектам, которые стоят раньше в случайной перестановке.

Ordered boosting: остаток объекта считается моделью, которая этот объект не видела. Так борются со сдвигом предсказаний (prediction shift).

Категориальные столбцы нужно указать в параметре cat_features.

LightGBM (Light Gradient-Boosting Machine)

Строит деревья, добавляя на каждом шаге один лист:

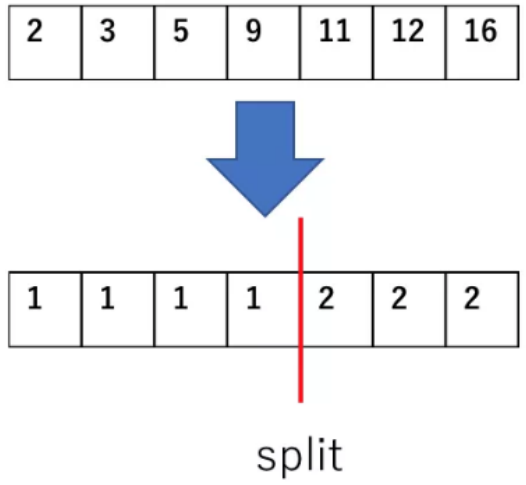


Источник: https://lightgbm.readthedocs.io/en/latest/Features.html

Такой подход позволяет добиться более высокой точности решения задач оптимизации

LightGBM (Light Gradient-Boosting Machine)

Ускоряет построение деревьев за счет биннинга: значения признаков сводятся к небольшому числу корзин, и разбиение ищется по гистограмме градиентов



Источник: https://towardsdatascience.com/

LightGBM (Light Gradient-Boosting Machine)

Ключевые особенности:

— как правило, деревья решений имеют несимметричную форму

— хорошо оптимизирован и не требует большого количества вычислительных ресурсов

— может работать с категориальными признаками и делает их разбиение на два подмножества достаточно эффективно за $O(k * \log k)$ операций

LightGBM: GOSS и EFB

GOSS (Gradient-based One-Side Sampling): оставляем все объекты с большим |градиентом|, из остальных берем случайную долю и увеличиваем ее вес.

EFB (Exclusive Feature Bundling): признаки, которые почти никогда не бывают ненулевыми одновременно (например, столбцы one-hot), склеиваются в один.

Обе идеи ускоряют обучение: меньше объектов и меньше признаков в гистограммах.

Сложность дерева задают num_leaves и min_data_in_leaf, а не только max_depth.

Реализации градиентного бустинга

	XGBoost	LightGBM	CatBoost
Общие характеристики	1. Алгоритм градиентного бустинга на деревьях решений 2. Жадные алгоритмы		
Преимущества	1. Параллельные вычисления 2. Вторая производная функции потерь 3. Поддержка линейных моделей 4. Регуляризация 5. Фактор усадки (shrinkage) 6. Выбор столбцов 7. Обработка пропусков	1. Алгоритм на основе гистограмм 2. Градиентный односторонний отбор (GOSS) 3. Эксклюзивное объединение признаков (EFB) 4. Алгоритм построения листьев 5. Параллельное ускорение обработки данных 6. Оптимизация кеша	1. Сокращает необходимость подбора гиперпараметров 2. Интерфейс для интеграции с scikit, а также с R и командной строкой 3. Поддержка категориальных признаков 4. Самообучающиеся функции потерь 5. Масштабируемая версия для GPU 6. Эффективное развертывание
Недостатки	1. Точный алгоритм проходит весь датасет при разбиении узлов 2. Предварительная сортировка дорогая (режим hist ее не требует)	1. Может создавать более глубокие деревья при построении по листьям, что вызывает переобучение 2. Чувствительность к шуму	1. Обработка категориальных признаков требует много памяти и времени 2. Настройки генератора случайных чисел влияют на конечный результат
Гиперпараметры	learning_rate, max_depth, min_child_weight, colsample_bytree, subsample, n_estimators	learning_rate, num_leaves, max_depth, min_data_in_leaf, categorical_feature, feature_fraction, bagging_fraction, num_iterations	learning_rate, depth, l2_leaf_reg, cat_features, one_hot_max_size, rsm, iterations

Источник: https://www.researchgate.net/figure/The-main-characteristics-and-important-parameters-of-XGBoost-LightGBM-and-CatBoost_tbl1_365435743

Реализации градиентного бустинга

Характеристика	XGBoost	LightGBM	CatBoost
Основное преимущество	Регуляризация, сильные значения по умолчанию	Очень быстрый на больших датасетах, экономит память	Нативная работа с категориальными признаками, меньше настройки
Построение дерева	По уровням (сбалансированный)	По листьям (быстрее, риск переобучения)	Симметричные деревья; ordered boosting против сдвига предсказаний
Работа с категориальными признаками	Нужно кодировать самим или включить enable_categorical	Нужно вручную указывать категориальные столбцы	Автоматически, через ordered target statistics (столбцы нужно указать в cat_features)
Скорость и использование памяти	Быстрый, использует больше памяти	Чрезвычайно быстрый и экономичный по памяти	Скорость на CPU умеренная, быстрое применение, есть обучение на GPU
Лучший сценарий применения	Малые и средние числовые датасеты	Критична высокая скорость, большие табличные данные	Смешанные типы данных, много категорий, автономная стабильная работа

Источник: https://medium.com/codex/gradient-boosting-xgboost-lightgbm-and-catboost-explained-3b90a753bf98

Параметры в трех библиотеках

Смысл	XGBoost	LightGBM	CatBoost
Число деревьев	n_estimators	n_estimators	iterations
Темп обучения	learning_rate	learning_rate	learning_rate
Глубина	max_depth	max_depth	depth
Число листьев	—	num_leaves	—
Доля объектов в дереве	subsample	subsample (нужен subsample_freq ≥ 1)	subsample (при bootstrap_type=Bernoulli)
Доля признаков	colsample_bytree	colsample_bytree	rsm
Минимум в листе	min_child_weight (сумма h)	min_child_samples	min_data_in_leaf (не для симметричных деревьев)
L2-штраф	reg_lambda	reg_lambda	l2_leaf_reg

Названия из sklearn-интерфейса. В «родном» интерфейсе LightGBM те же параметры называются feature_fraction, bagging_fraction, min_data_in_leaf, num_iterations.

Документация

- XGBoost
- CatBoost
- LightGBM